

RELAZIONE PROGETTO – SKILLSWAP SCHOOL

Introduzione

SkillSwap School è un progetto sviluppato in Java che simula una piattaforma di scambio competenze tra studenti. L'obiettivo principale del sistema è permettere agli utenti di offrire competenze che conoscono bene e richiedere aiuto in altre materie o abilità. Il programma si occupa quindi di gestire studenti, skill, offerte, richieste, match automatici, scambi e recensioni.

Il progetto è stato sviluppato utilizzando la programmazione orientata agli oggetti e organizzando il codice in package separati per rendere il sistema più ordinato, leggibile e facile da mantenere.

Struttura del progetto

Il progetto è diviso in quattro package principali:

- domain
- service
- storage
- app

Package domain

Il package `domain` contiene tutte le classi che rappresentano le entità principali del sistema.

Le classi più importanti sono:

- Student
- Skill
- Offer
- Request
- Exchange
- Review

- **MatchResult**

Student

La classe **Student** rappresenta uno studente registrato nella piattaforma. Contiene:

- id
- nome
- classe
- email
- rating medio
- numero recensioni

È presente anche il metodo **addRating()** che aggiorna automaticamente la media delle recensioni ricevute.

Skill

La classe **Skill** rappresenta una competenza. Ogni skill possiede:

- id
- nome
- categoria

Le categorie disponibili sono:

- SUBJECT
- LANGUAGE
- SOFT_SKILL
- TECH

Offer e Request

Offer rappresenta una competenza che uno studente offre agli altri utenti.

Request rappresenta invece una richiesta di aiuto su una determinata skill.

Entrambe le classi contengono:

- studente
- skill
- livello
- nota

Exchange

La classe **Exchange** rappresenta uno scambio tra due studenti. Ogni exchange contiene:

- un'offerta
- una richiesta
- uno stato
- data creazione
- data chiusura

Gli stati disponibili sono:

- PROPOSED
- ACCEPTED
- COMPLETED
- CANCELLED

Review

La classe **Review** rappresenta una recensione lasciata al termine di uno scambio. Contiene:

- reviewer
- reviewee
- stelle
- commento

MatchResult

Questa classe viene utilizzata dal sistema di matching per rappresentare un match trovato tra studenti. Contiene:

- studente trovato
- offer
- request
- score
- motivazione del match

Package service

Nel package **service** è presente tutta la logica applicativa del progetto.

Le classi principali sono:

- **SkillSwapService**

- MatchingService
- ExchangeService
- ReviewService

SkillSwapService

È il servizio principale del progetto e gestisce:

- registrazione studenti
- aggiunta skill
- creazione offer
- creazione request
- visualizzazione dati

Questo servizio viene utilizzato direttamente dalla console principale.

È presente anche il metodo `seedSkills()` che carica automaticamente alcune skill iniziali nel sistema.

MatchingService

`MatchingService` è una delle parti più importanti del progetto. Si occupa di trovare studenti compatibili.

Sono stati implementati due tipi di matching:

One-way matching

Il sistema cerca uno studente che possa soddisfare una richiesta.

Swap matching

Il sistema controlla anche che esista uno scambio reciproco tra i due studenti. Per

rendere il matching più realistico è stato implementato un sistema a punteggio:

- +3 se la skill coincide
- +2 se il livello è sufficiente
- +1 se gli studenti appartengono alla stessa classe

I risultati vengono ordinati in base allo score.

Per evitare risultati duplicati nello swap matching è stato utilizzato un `HashSet`.

ExchangeService

`ExchangeService` gestisce il ciclo di vita degli exchange.

Le operazioni principali sono:

- `propose()`
- `accept()`
- `complete()`
- `cancel()`

Il servizio controlla:

- validità delle offerte
- esistenza delle richieste
- stato corretto degli exchange

Una scelta importante è stata centralizzare tutta la gestione degli stati in un solo servizio per mantenere il controllo del flusso dell'applicazione.

ReviewService

`ReviewService` gestisce le recensioni.

Una review può essere inserita solamente:

- se l'exchange è COMPLETED
- da uno dei due studenti coinvolti
- una sola volta per studente

Quando una review viene aggiunta:

- il rating medio dello studente recensito viene aggiornato automaticamente

È presente anche un controllo per impedire recensioni duplicate.

Package storage

Il package `storage` gestisce la persistenza dei dati.

Le classi principali sono:

- **Storage**
- **InMemoryStorage**
- **FileStorage**
- **SkillSwapState**

SkillSwapState

Questa classe contiene tutte le strutture dati principali del sistema:

- studenti
- skill
- offer
- request
- exchange
- review

Funziona come memoria centrale dell'applicazione.

InMemoryStorage

InMemoryStorage salva i dati solamente in memoria. È

stato utilizzato principalmente:

- durante lo sviluppo
- nei test

FileStorage

FileStorage permette di salvare e caricare i dati tramite file CSV. I

file utilizzati sono:

- students.csv
- skills.csv
- offers.csv
- requests.csv

- exchanges.csv
- reviews.csv

Durante il caricamento dei dati è stato necessario ricostruire i collegamenti tra le entità utilizzando gli ID salvati nei file.

Ad esempio:

- un **Offer** deve recuperare lo **Student** e la **Skill**
- un **Exchange** deve recuperare **Offer** e **Request**

Package app

Il package **app** contiene la parte console del progetto.

Le classi principali sono:

- **SkillSwapApp**
- **ConsoleReportPrinter**

SkillSwapApp

Gestisce:

- menu principale
- input utente
- chiamate ai servizi

ConsoleReportPrinter

Serve per stampare:

- profili studenti
- match
- exchange
- leaderboard

La classe è stata creata per migliorare la leggibilità dell'output console.

Testing

Per verificare il corretto funzionamento del progetto sono stati implementati 12 test principali. I test controllano:

- matching
- swap matching
- validazione livelli
- gestione exchange
- review
- review duplicate
- validazione studenti

Nei test è stato utilizzato **InMemoryStorage** per evitare modifiche ai file CSV reali.

Scelte progettuali

Una delle scelte principali è stata separare:

- dominio
- logica applicativa
- persistenza

Questo approccio rende il progetto:

- più ordinato
- più leggibile
- più semplice da modificare

Un'altra scelta importante è stata creare servizi separati per:

- matching
- exchange
- review

In questo modo ogni classe ha una responsabilità precisa.

Conclusione

Il progetto ha permesso di applicare diversi concetti importanti della programmazione orientata agli oggetti:

- organizzazione a package
- utilizzo collezioni
- persistenza su file
- testing
- separazione responsabilità

Il sistema potrebbe essere ampliato in futuro aggiungendo:

- database reale
- interfaccia grafica
- autenticazione utenti
- matching più avanzato.